

TalentGeenie — Low-Level Design (LLD)

Document Version: 2.0
Date: March 5, 2026
Product: TalentGeenie — AI-Powered Interview & Learning Platform
Classification: Internal / Confidential

Table of Contents

- 1. [Introduction](#)
- 2. [Frontend Architecture Detail](#)
- 3. [Backend Service Detail](#)
- 4. [Database Schema Detail](#)
- 5. [API Design](#)
- 6. [Core Flow Sequence Diagrams](#)
- 7. [State Management Detail](#)
- 8. [Component Design](#)
- 9. [Security Implementation Detail](#)
- 10. [Error Handling Architecture](#)
- 11. [Proctoring Engine Detail](#)
- 12. [File Upload Architecture](#)
- 13. [Configuration Management](#)
- 14. [Testing Architecture](#)

1. Introduction

1.1 Purpose

This Low-Level Design (LLD) document provides the detailed technical design for every component of the TalentGeenie platform. It covers class-level design, data structures, algorithms, API contracts, sequence diagrams, and implementation details.

1.2 References

Document	Location
Requirements (SRS)	docs/REQUIREMENTS_DOCUMENT.md
High-Level Design (HLD)	docs/HIGH_LEVEL_DESIGN.md

2. Frontend Architecture Detail

2.1 Application Entry Point & Provider Hierarchy

```
flowchart TD
    A["index.html"] --> B["main.tsx"]
    B --> C["React.StrictMode"]
    C --> D["BrowserRouter"]
    D --> E["App.tsx"]
```

```

    E --> F["QueryClientProvider<br/>TanStack React Query<br/>(cache, retry,
devtools)"]
    E --> G["AuthProvider<br/>AuthContext<br/>(user, session, roles)"]
    E --> H["OrganizationProvider<br/>OrgContext<br/>(selectedOrg, impersonation)"]
    E --> I["Toaster (Sonner notifications)"]
    E --> J["ErrorBoundary (Sentry reporting)"]
    E --> K["Routes (React Router v6)"]
    K --> L["Public routes"]
    K --> M["Protected routes"]
    K --> N["Catch-all 404"]
    M --> O["AdminLayout → /admin/*"]
    M --> P["PartnerLayout → /partner/*"]
    M --> Q["Common pages"]

style A fill:#e1bee7,stroke:#6a1b9a
style E fill:#bbdefb,stroke:#1565c0
style K fill:#c8e6c9,stroke:#2e7d32

```

2.2 Routing Architecture

```

flowchart TD
    ROOT["src/App.tsx – Route Definitions"]

    ROOT --> PUBLIC
    ROOT --> PROTECTED
    ROOT --> ADMIN
    ROOT --> PARTNER
    ROOT --> RECRUITING

    subgraph PUBLIC["PUBLIC ROUTES (No authentication)"]
        P1["/ → Landing"]
        P2["/auth → Auth (login/register)"]
        P3["/auth/verify → AuthVerify"]
        P4["/auth/verify-email → VerifyEmail"]
        P5["/reset-password → ResetPassword"]
        P6["/reset-password/confirm → ResetPasswordConfirm"]
        P7["/take-interview/:shareLink → TakeInterview (lazy)"]
        P8["/i/:orgSlug/:slug/:token → TakeInterview (lazy)"]
        P9["/interview-complete/:id → InterviewComplete"]
        P10["/pricing → Pricing"]
        P11["/learning → Certifications"]
        P12["/certifications → Certifications"]
        P13["/verify-certificate → VerifyCertificate"]
        P14["/partner/onboarding → PartnerOnboarding"]
    end

    end

    subgraph PROTECTED["PROTECTED ROUTES (Auth required)"]
        PR1["/dashboard → RoleBasedRedirect"]
        PR2["/profile → Profile"]
        PR3["/notifications → Notifications"]
        PR4["/settings → Settings"]
    end

```

```

    PR5["/my-applications → MyApplications"]
    PR6["/learning-dashboard → LearningDashboard"]
    PR7["/my-learning-plan → MyLearningPlan"]
    PR8["/my-certificates → MyCertificates"]
end

subgraph ADMIN["ADMIN ROUTES (/admin/*)<br/>Wrapper: AdminLayout –
platform_admin only"]
    A1["/admin → PlatformAdminHub"]
    A2["/admin/organizations → OrganizationsList"]
    A3["/admin/user-management → UnifiedUserManagement"]
    A4["/admin/applications → PartnerApplicationsReview"]
    A5["/admin/role-assignment → RoleAssignment"]
    A6["/admin/role-permissions → RolePermissionsManagement"]
    A7["/admin/analytics → AdvancedAnalytics"]
    A8["/admin/settings → Settings"]
    A9["/admin/billing → BillingManagement"]
    A10["/admin/training → AdminTraining"]
    A11["/admin/learning-management → PlatformAdminLearning"]
    A12["/admin/certification-admin → CertificationAdmin"]
    A13["/admin/certification-analytics → CertificationAnalytics"]
    A14["/admin/certification-configuration → CertificationConfiguration"]
    A15["/admin/ai-configuration → AIConfiguration"]
    A16["/admin/chatbot-management → ChatbotManagement"]
    A17["/admin/testing-hub → TestingHub"]
    A18["/admin/deploy → DeploymentConfigurator"]
    A19["/admin/plan-management → PlanManagement"]
    A20["/admin/promotions → PromotionManagement"]
    A21["/admin/learning-plan-management → LearningPlanManagement"]
    A22["/admin/payment-gateways → PaymentGatewayManagement"]
    A23["/admin/email-configuration → EmailConfiguration"]
    A24["/admin/preinterview-logs → PreInterviewCheckLogs"]
    A25["/admin/operation-logs → InterviewOperationLogs"]
    A26["/admin/log-analysis → LogAnalysis"]
    A27["/admin/cost-monitoring → PartnerCostMonitoring"]
    A28["/admin/scheduled-jobs → ScheduledJobsAdmin"]
    A29["/admin/ai-usage-monitoring → AIUsageMonitoring"]
    A30["/admin/system-monitoring → SystemMonitoring"]
end

subgraph PARTNER["PARTNER ROUTES (/partner/*)<br/>Wrapper:
PartnerLayout<br/>Roles: partner_admin, platform_admin, hr_recruiter, tech_spoc,
billing_contact"]
    PA1["/partner → PartnerPortal"]
    PA2["/partner/portal → PartnerPortal"]
    PA3["/partner/dashboard → UnifiedDashboard"]
    PA4["/partner/settings → OrganizationSettings"]
    PA5["/partner/analytics → OrganizationAnalytics"]
    PA6["/partner/users → UnifiedUserManagement"]
    PA7["/partner/billing → PartnerBilling"]
    PA8["/partner/reports → PartnerReports"]
end

```

```

    subgraph RECRUITING["RECRUITING ROUTES (/partner/recruiting/*)<br/>Wrapper:
PartnerLayout<br/>Roles: hr_recruiter, tech_spoc, partner_admin, platform_admin"]
        R1["/partner/recruiting/jd-builder → JDBuilderWizard"]
        R2["/partner/recruiting/create-interview → CreateInterview"]
        R3["/partner/recruiting/quick-create → QuickCreatePreview"]
        R4["/partner/recruiting/interviews → Dashboard"]
        R5["/partner/recruiting/interview/:id → InterviewDetail"]
        R6["/partner/recruiting/assessment/:id → AssessmentReport"]
        R7["/partner/recruiting/proctoring → ProctoringDashboard"]
        R8["/partner/recruiting/proctoring-settings → ProctoringSettings"]
        R9["/partner/recruiting/question-repository → QuestionRepository"]
        R10["/partner/recruiting/templates → TemplatesLibrary"]
        R11["/partner/recruiting/report-builder → ReportBuilder"]
    end

    style PUBLIC fill:#e8f5e9,stroke:#2e7d32
    style PROTECTED fill:#e3f2fd,stroke:#1565c0
    style ADMIN fill:#fce4ec,stroke:#c62828
    style PARTNER fill:#fff3e0,stroke:#e65100
    style RECRUITING fill:#f3e5f5,stroke:#6a1b9a

```

2.3 Layout Component Architecture

```

flowchart TD
    subgraph AppLayout["AppLayout"]
        subgraph Navbar["AppNavbar (data-component=app-navbar)"]
            Logo["Logo"]
            Nav["Navigation<br/>(Role-based)"]
            Search["Search<br/>(Cmd+K)"]
            Profile["Profile Menu"]
        end
        Banner["ImpersonationBanner<br/>(shown when admin impersonates org)"]
        Breadcrumbs["RoleBreadcrumbs<br/>(context-aware path)"]
        Outlet["Outlet – Page Content"]
    end

    AppLayout --> AdminLayout["AdminLayout<br/>Checks: platform_admin<br/>Redirects non-admin → /dashboard"]
    AppLayout --> PartnerLayout["PartnerLayout<br/>Checks: partner_admin, hr_recruiter,<br/>tech_spoc, billing_contact, platform_admin<br/>Loads organization context"]

    style AppLayout fill:#e3f2fd,stroke:#1565c0
    style Navbar fill:#fff3e0,stroke:#e65100
    style Banner fill:#fce4ec,stroke:#c62828
    style Breadcrumbs fill:#e8f5e9,stroke:#2e7d32
    style Outlet fill:#f3e5f5,stroke:#6a1b9a
    style AdminLayout fill:#fce4ec,stroke:#c62828
    style PartnerLayout fill:#fff3e0,stroke:#e65100

```

2.4 ProtectedRoute Component Design

```
// src/components/ProtectedRoute.tsx

interface ProtectedRouteProps {
  children: React.ReactNode;
  requiredRoles?: AppRole[];
  requiredPermissions?: Permission[];
  redirectTo?: string;
}

// Decision tree:
//
// Is user authenticated?
//   NO → redirect to /auth
//   YES → Is email verified? (if required)
//         NO → show VerifyEmail
//         YES → Has required role?
//               NO → show AccessDenied / redirect
//               YES → Has required permission?
//                     NO → show AccessDenied
//                     YES → render children
```

2.5 Lazy Loading Strategy

```
flowchart TD
    APP["React.lazy Loading Strategy"]

    subgraph Eager["EAGERLY LOADED – main bundle"]
        E1["Landing, Auth, Profile, Dashboard"]
        E2["Navigation, Layout components"]
        E3["All UI primitives (shadcn/ui)"]
        E4["Contexts, Hooks, Utils"]
    end

    subgraph Lazy["LAZILY LOADED – code-split chunks<br/>Wrapped in: Suspense<br/>fallback=PageLoader"]
        L1["TakeInterview<br/>(MediaPipe, recording, timer)"]
        L2["TakeLearningAssessment<br/>(similar to TakeInterview)"]
        L3["TakeCertification<br/>(proctoring + exam engine)"]
        L4["ProctoringDashboard<br/>(video players, reports)"]
        L5["ProctoringSettings<br/>(configuration UI)"]
        L6["ProctoringTestPage<br/>(testing tools)"]
    end

    APP --> Eager
    APP --> Lazy
```

```
style Eager fill:#e8f5e9,stroke:#2e7d32
style Lazy fill:#fff3e0,stroke:#e65100
```

3. Backend Service Detail

3.1 Supabase Services Architecture

```
flowchart TD
    Traffic["External Traffic"] --> Kong
    subgraph Kong["KONG API GATEWAY (Port 8000)"]
        R1["/auth/v1/* → GoTrue (9999)"]
        R2["/rest/v1/* → PostgREST (3000)"]
        R3["/functions/v1/* → Functions (9000)"]
        R4["/realtime/v1/* → Realtime (4000)"]
        R5["/storage/v1/* → Storage (5000)"]
        AuthH["Auth: Authorization Bearer JWT<br/>API key: anon_key | service_role_key"]
    end
    Kong --> GoTrue
    Kong --> PostgREST
    subgraph GoTrue["GoTrue – Auth Service"]
        GA["Endpoints:<br/>POST /signup, /token, /recover<br/>POST /verify, /logout<br/>GET /user, PUT /user<br/><br/>Config:<br/>• JWT secret<br/>• SMTP settings<br/>• Auto-confirm<br/>• Email templates"]
    end
    subgraph PostgREST["PostgREST – REST API"]
        PR["Auto-generated REST from schema<br/>GET/POST/PATCH/DELETE /table<br/>POST /rpc/fn<br/>Uses RLS for all queries<br/>Connects via PgBouncer"]
    end
    GoTrue --> PgBouncer
    PostgREST --> PgBouncer
    subgraph PgBouncer["PgBouncer (Port 6432)"]
        PB["Mode: transaction<br/>Pool Size: 20 per user<br/>Max Client Connections: 1000<br/><br/>CRITICAL: prepared statement errors<br/>require container restart when<br/>stale connections accumulate"]
    end
    PgBouncer --> PostgreSQL
    subgraph PostgreSQL["PostgreSQL 15.8 (Port 5432)"]
        PG["Database: postgres<br/>Schemas: public, auth, storage, extensions<br/><br/>Key Settings:<br/>• max_connections: 200<br/>• shared_buffers: 256MB<br/>• effective_cache_size: 1GB<br/>• wal_level: replica"]
    end
```

```

style Kong fill:#fff3e0,stroke:#e65100
style GoTrue fill:#e3f2fd,stroke:#1565c0
style PostgREST fill:#e8f5e9,stroke:#2e7d32
style PgBouncer fill:#fce4ec,stroke:#c62828
style PostgreSQL fill:#f3e5f5,stroke:#6a1b9a

```

3.2 Edge Function Architecture

```

flowchart TD
    ROOT["supabase/functions/"] --> SHARED
    ROOT --> GQ
    ROOT --> EI
    ROOT --> CUS
    ROOT --> IPS
    ROOT --> SII
    ROOT --> MORE["100+ more functions..."]

    subgraph SHARED["_shared/ - Shared utilities"]
        S1["cors.ts - CORS headers"]
        S2["supabase-client.ts - Service role client"]
        S3["validation.ts - Input validation"]
    end

    subgraph GQ["generate-questions/ - AI question generation"]
        GQ1["Input: interviewId, config"]
        GQ2["Process:<br/>1. Fetch interview from DB<br/>2. Build prompt from JD + config<br/>3. Call Gemini 2.5 Flash API<br/>4. Parse structured JSON response<br/>5. Validate question count/types/difficulty<br/>6. Insert questions into DB"]
        GQ3["Output: questions[], count"]
        GQ1 --> GQ2 --> GQ3
    end

    subgraph EI["evaluate-interview/ - AI candidate evaluation"]
        EI1["Input: attemptId"]
        EI2["Process:<br/>1. Fetch attempt + questions + answers<br/>2. Build evaluation prompt<br/>3. Call Gemini 2.5 Flash API<br/>4. Parse scores per question<br/>5. Calculate overall score + CPI<br/>6. Determine hiring decision<br/>7. Insert assessment record"]
        EI3["Output: assessment"]
        EI1 --> EI2 --> EI3
    end

    subgraph CUS["complete-user-signup/ - Post-auth setup"]
        CUS1["Input: userId, email"]
        CUS2["Process:<br/>1. Check if profile exists<br/>2. Create profile (email_verified: false)<br/>3. Insert guest role into user_roles<br/>4. Initialize onboarding_progress<br/>5. Send welcome email"]
        CUS3["Output: success boolean"]
        CUS1 --> CUS2 --> CUS3
    end

```

3.3 Edge Function Invocation Pattern

4. Database Schema Detail

profiles Table[illegible]

```

avatar_url TEXT,
phone      TEXT,
bio        TEXT,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now(),

-- Email verification
email_verified BOOLEAN DEFAULT false,

CONSTRAINT profiles_email_check CHECK (length(email) <= 255),
CONSTRAINT profiles_name_check CHECK (
    full_name IS NULL OR (
        length(full_name) >= 2 AND
        length(full_name) <= 100 AND
        full_name ~ '^[a-zA-Z\s\''-~]+$'
    )
);

-- RLS Policies:
-- SELECT: Users can read own profile; platform_admin reads all
-- UPDATE: Users can update own profile only
-- INSERT: Triggered by complete-user-signup edge function

```

user_roles Table

```

CREATE TABLE user_roles (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    role        app_role NOT NULL,
    assigned_by UUID REFERENCES profiles(id),
    assigned_at TIMESTAMPTZ DEFAULT now(),
    is_active   BOOLEAN DEFAULT true,

    UNIQUE(user_id, role)
);

-- app_role enum values:
-- 'platform_admin', 'partner_admin', 'hr_recruiter',
-- 'tech_spoc', 'billing_contact', 'guest',
-- Legacy: 'admin', 'hr', 'interviewer', 'contributor',
--         'candidate', 'ta_creator'

```

organizations Table

```

CREATE TABLE organizations (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        TEXT NOT NULL,
    slug        TEXT UNIQUE NOT NULL,
    logo_url    TEXT,

```

```

industry      TEXT,
size          TEXT,          -- 'startup', 'small', 'medium', 'large',
'enterprise'
pricing_model TEXT,
settings      JSONB DEFAULT '{}',
is_active     BOOLEAN DEFAULT true,
created_at    TIMESTAMPTZ DEFAULT now(),
updated_at    TIMESTAMPTZ DEFAULT now(),
created_by    UUID REFERENCES profiles(id)
);

```

interviews Table

```

CREATE TABLE interviews (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title             TEXT NOT NULL,          -- 5-200 chars
  job_title         TEXT,
  job_description   TEXT NOT NULL,          -- 50-10,000 chars
  status            TEXT DEFAULT 'active', -- 'draft', 'active', 'archived'
  question_count    INTEGER NOT NULL,       -- 5-100
  time_limit        INTEGER,                -- 0-180 minutes
  share_link        TEXT UNIQUE,
  skills            TEXT[],                 -- Extracted skills array
  difficulty_distribution JSONB,             -- {easy: %, medium: %, hard: %}
  question_type_config JSONB,               -- {mcq: n, scenario: n, coding: n,
descriptive: n}
  topic_distribution JSONB,                 -- {topic: percentage, ...}
  proctoring_enabled BOOLEAN DEFAULT false,
  expires_at        TIMESTAMPTZ,
  organization_id    UUID REFERENCES organizations(id),
  created_by         UUID REFERENCES profiles(id),
  created_at         TIMESTAMPTZ DEFAULT now(),
  updated_at         TIMESTAMPTZ DEFAULT now(),

  CONSTRAINT interviews_title_length CHECK (length(title) >= 5 AND length(title) <=
200),
  CONSTRAINT interviews_jd_length CHECK (length(job_description) >= 50),
  CONSTRAINT interviews_question_count CHECK (question_count >= 5 AND question_count
<= 100)
);

```

questions Table

```

CREATE TABLE questions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  interview_id       UUID NOT NULL REFERENCES interviews(id) ON DELETE CASCADE,
  question_text      TEXT NOT NULL,
  question_type      TEXT NOT NULL,        -- 'mcq', 'scenario', 'coding', 'descriptive'
  difficulty         TEXT NOT NULL,        -- 'easy', 'medium', 'hard'
  topic             TEXT,

```

```

options          JSONB,          -- MCQ: [{label, value}]
correct_answer   TEXT,          -- HIDDEN from candidates via RLS
explanation        TEXT,
code_template     TEXT,          -- For coding questions
test_cases       JSONB,          -- For coding questions
is_approved      BOOLEAN DEFAULT true,
display_order     INTEGER,
created_at       TIMESTAMPTZ DEFAULT now()
);

-- RLS: correct_answer column hidden from candidates
-- Only visible to interview creator / org members with view access

```

interview_attempts Table

```

CREATE TABLE interview_attempts (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  interview_id      UUID NOT NULL REFERENCES interviews(id),
  candidate_email   TEXT NOT NULL,      -- IMMUTABLE after creation
  candidate_name    TEXT NOT NULL,      -- IMMUTABLE after creation
  session_token     TEXT NOT NULL,      -- 64-char base64, stored in sessionStorage
  answers           JSONB DEFAULT '{}', -- {question_uuid: "answer text"}
  status            TEXT DEFAULT 'in_progress', -- 'in_progress', 'completed',
'timed_out'
  time_taken        INTEGER,            -- seconds (0-86,400)
  started_at        TIMESTAMPTZ DEFAULT now(),
  completed_at      TIMESTAMPTZ,

  -- Prevents duplicate attempts
  UNIQUE(interview_id, candidate_email),

  -- PII immutability enforced by WITH CHECK:
  -- UPDATE cannot change candidate_email or candidate_name
);

```

assessments Table

```

CREATE TABLE assessments (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id        UUID NOT NULL REFERENCES interview_attempts(id),
  overall_score      NUMERIC(5,2),      -- 0.00-100.00
  hiring_decision    TEXT,              -- 'strong_hire', 'hire', 'lean_hire',
'no_hire'
  strengths          TEXT[],
  weaknesses         TEXT[],
  topic_scores       JSONB,            -- {topic: {score, analysis}}
  detailed_analysis  JSONB,            -- Per-question analysis
  cpi_technical      NUMERIC(5,2),
  cpi_problem_solving NUMERIC(5,2),
  cpi_communication  NUMERIC(5,2),

```

```

    cpi_integrity    NUMERIC(5,2),
    created_at       TIMESTAMPTZ DEFAULT now(),

    UNIQUE(attempt_id)
);

```

4.2 Proctoring Tables

proctoring_sessions Table

```

CREATE TABLE proctoring_sessions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id        UUID NOT NULL REFERENCES interview_attempts(id),
  video_url          TEXT,
  screen_recording_url TEXT,
  violations          JSONB DEFAULT '[]',
  integrity_score    NUMERIC(5,2),    -- 0-100
  tab_switch_count   INTEGER DEFAULT 0,
  look_away_count    INTEGER DEFAULT 0,
  face_detection_data JSONB,
  eye_tracking_data  JSONB,
  voice_analysis_data JSONB,
  review_status      TEXT DEFAULT 'pending', -- 'pending', 'reviewed', 'flagged'
  upload_status      TEXT DEFAULT 'pending', -- 'pending', 'uploading',
  'complete', 'failed'
  created_at         TIMESTAMPTZ DEFAULT now(),
  updated_at         TIMESTAMPTZ DEFAULT now()
);

```

4.3 Billing Tables

```

-- subscription_plans
CREATE TABLE subscription_plans (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name              TEXT NOT NULL,
  price_monthly_cents INTEGER NOT NULL,    -- In cents (e.g., 9900 = $99.00)
  max_interviews    INTEGER,
  max_users          INTEGER,
  max_ai_usage       INTEGER,              -- AI tokens/month
  features           JSONB DEFAULT '{}',
  is_active          BOOLEAN DEFAULT true,
  created_at         TIMESTAMPTZ DEFAULT now()
);

-- organization_subscriptions
CREATE TABLE organization_subscriptions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id    UUID REFERENCES organizations(id),
  plan_id            UUID REFERENCES subscription_plans(id),
  stripe_subscription_id TEXT,

```

```

stripe_customer_id TEXT,
status             TEXT DEFAULT 'active', -- 'active', 'past_due', 'cancelled'
current_period_start TIMESTAMPTZ,
current_period_end  TIMESTAMPTZ,
interviews_used     INTEGER DEFAULT 0,
ai_tokens_used      INTEGER DEFAULT 0,
created_at          TIMESTAMPTZ DEFAULT now()
);

```

4.4 Learning Tables

```

-- certification_topics
CREATE TABLE certification_topics (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  provider    TEXT,                -- 'azure', 'aws', 'gcp', 'databricks',
  'snowflake'
  difficulty  TEXT DEFAULT 'intermediate',
  passing_score INTEGER DEFAULT 70,
  syllabus    JSONB,
  time_limit  INTEGER DEFAULT 60, -- minutes
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- user_certificates
CREATE TABLE user_certificates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES profiles(id),
  certification_id UUID REFERENCES certification_topics(id),
  attempt_id  UUID REFERENCES certification_attempts(id),
  verification_code TEXT UNIQUE NOT NULL,
  score       NUMERIC(5,2),
  issued_at   TIMESTAMPTZ DEFAULT now(),
  expires_at  TIMESTAMPTZ
);

```

4.5 Database Functions

```

-- Role checking (SECURITY DEFINER, SET search_path = public)
CREATE FUNCTION has_role(uid UUID, required_role app_role)
RETURNS BOOLEAN AS $$
  SELECT EXISTS (
    SELECT 1 FROM user_roles
    WHERE user_id = uid AND role = required_role AND is_active = true
  );
$$ LANGUAGE sql SECURITY DEFINER;

-- Secure interview creation for candidate

```

```

CREATE FUNCTION create_interview_attempt(
    p_interview_id UUID,
    p_candidate_email TEXT,
    p_candidate_name TEXT
) RETURNS TABLE(attempt_id UUID, session_token TEXT) AS $$
DECLARE
    v_token TEXT;
    v_attempt_id UUID;
BEGIN
    -- Generate 64-char base64 session token
    v_token := encode(gen_random_bytes(48), 'base64');

    INSERT INTO interview_attempts (interview_id, candidate_email, candidate_name,
session_token)
VALUES (p_interview_id, p_candidate_email, p_candidate_name, v_token)
RETURNING id INTO v_attempt_id;

    RETURN QUERY SELECT v_attempt_id, v_token;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Questions retrieval (hides correct_answer)
CREATE FUNCTION get_questions_for_candidate(p_attempt_id UUID)
RETURNS TABLE(
    id UUID, question_text TEXT, question_type TEXT,
    difficulty TEXT, topic TEXT, options JSONB,
    code_template TEXT, display_order INTEGER
) AS $$
SELECT q.id, q.question_text, q.question_type,
       q.difficulty, q.topic, q.options,
       q.code_template, aq.display_order
FROM attempt_questions aq
JOIN questions q ON q.id = aq.question_id
WHERE aq.attempt_id = p_attempt_id
ORDER BY aq.display_order;
-- NOTE: correct_answer is NOT returned
$$ LANGUAGE sql SECURITY DEFINER;

-- API key encryption
CREATE FUNCTION encrypt_api_key(plain_key TEXT, encryption_key TEXT)
RETURNS TEXT AS $$
SELECT encode(
    encrypt(plain_key::bytea, encryption_key::bytea, 'aes'),
    'base64'
);
$$ LANGUAGE sql SECURITY DEFINER;

```

4.6 Entity Relationship Diagram (Detailed)

```

erDiagram
    AUTH_USERS ||--|| PROFILES : "1:1"

```

```
PROFILES ||--o{ USER_ROLES : "1:N"
PROFILES }o--o{ ORG_MEMBERS : "N:M"
PROFILES ||--o{ NOTIFICATIONS : "1:N"
ORG_MEMBERS }o--|| ORGANIZATIONS : "N:1"
ORGANIZATIONS ||--o{ INTERVIEWS : "1:N"
ORGANIZATIONS ||--o{ ORG_SUBSCRIPTIONS : "1:N"
ORGANIZATIONS ||--o{ USAGE_TRACKING : "1:N"
ORG_SUBSCRIPTIONS }o--|| SUBSCRIPTION_PLANS : "N:1"
INTERVIEWS ||--o{ QUESTIONS : "1:N"
INTERVIEWS ||--o{ INVITATIONS : "1:N"
INTERVIEWS ||--o{ ATTEMPTS : "1:N"
ATTEMPTS ||--|| ASSESSMENTS : "1:1"
ATTEMPTS ||--|| PROCTORING_SESSIONS : "1:1"
```

```
AUTH_USERS {
    uuid id PK
    string email
    string encrypted_pw
}
PROFILES {
    uuid id "PK/FK"
    string email
    string full_name
    string avatar_url
    string phone
    string bio
    boolean email_verified
}
USER_ROLES {
    enum role
    boolean is_active
}
ORG_MEMBERS {
    uuid org_id
    uuid user_id
    string status
}
NOTIFICATIONS {
    string title
    string body
    boolean is_read
    string type
}
ORGANIZATIONS {
    string name
    string slug
    string industry
    string size
    jsonb settings
    boolean is_active
}
INTERVIEWS {
```

```
    string title
    text job_description
    string status
    int question_cnt
    int time_limit
    boolean proctoring
    text_arr skills
    uuid org_id
    uuid created_by
}
ORG_SUBSCRIPTIONS {
    uuid plan_id
    string stripe_sub_id
    string status
}
USAGE_TRACKING {
    int interviews_used
    int ai_tokens_used
    string period
}
SUBSCRIPTION_PLANS {
    string name
    decimal price
    int max_interviews
    int max_users
}
QUESTIONS {
    text question_text
    string type
    string difficulty
    string topic
    jsonb options
    text answer "hidden via RLS"
}
INVITATIONS {
    string candidate_email
    string share_token
    string status
    timestamp expires_at
}
ATTEMPTS {
    string candidate_email
    string candidate_name
    string session_token
    jsonb answers
    string status
    int time_taken
}
ASSESSMENTS {
    float overall_score
    string hiring_decision
    text_arr strengths
}
```

```

        text_arr weaknesses
        jsonb topic_scores
        float cpi
    }
    PROCTORING_SESSIONS {
        string video_url
        jsonb violations
        float integrity_score
        int tab_switch_count
        string review_status
    }
}

```

5. API Design

5.1 REST API (PostgREST — Auto-generated)

All tables are exposed via PostgREST with RLS filtering:

BASE: `http://localhost:8000/rest/v1`

Headers:

```

apikey: <anon_key>
Authorization: Bearer <jwt>
Content-Type: application/json
Prefer: return=representation (for INSERT/UPDATE)

```

```

GET    /interviews?organization_id=eq.<uuid>&status=eq.active&order=created_at.desc
GET    /interviews?id=eq.<uuid>&select=*,questions(*),interview_invitations(*)
POST   /interviews                { title, job_description, ... }
PATCH /interviews?id=eq.<uuid>  { status: 'archived' }
DELETE /interviews?id=eq.<uuid>

```

```

GET    /profiles?id=eq.<uuid>
PATCH /profiles?id=eq.<uuid>  { full_name, phone, bio }

```

```

GET    /notifications?select=*&order=created_at.desc&is_read=eq.false
PATCH /notifications?id=eq.<uuid>  { is_read: true }

```

```

POST   /rpc/create_interview_attempt      { p_interview_id, p_email, p_name }
POST   /rpc/get_questions_for_candidate { p_attempt_id }
POST   /rpc/update_attempt_with_session { p_attempt_id, p_session_token,
p_answers }
POST   /rpc/has_role                  { uid, required_role }
POST   /rpc/can_user_retake_certification { user_id, cert_id }

```

5.2 Edge Function API

BASE: `http://localhost:8000/functions/v1`

Headers:

```
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
POST /generate-questions
Body: { interviewId: string, config?: object }
Response: { questions: Question[], count: number }
```

```
POST /evaluate-interview
Body: { attemptId: string }
Response: { assessment: Assessment }
```

```
POST /send-interview-invitations
Body: { interviewId: string, candidates: [{email, name}] }
Response: { sent: number, failed: number }
```

```
POST /init-proctoring-session
Body: { attemptId: string, checks: PreInterviewChecks }
Response: { sessionId: string, uploadUrls: {...} }
```

```
POST /generate-job-description
Body: { jobTitle: string, industry?: string, requirements?: string[] }
Response: { description: string, skills: string[] }
```

```
POST /complete-user-signup
Body: { userId: string, email: string }
Response: { success: boolean }
```

```
POST /generate-certificate-pdf
Body: { certificateId: string }
Response: { pdfUrl: string }
```

5.3 Realtime Subscriptions

```
// Notification subscription
supabase
  .channel('notifications')
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'notifications',
    filter: `user_id=eq.${userId}`
  }, (payload) => {
    // Show toast notification
  })
  .subscribe();

// Interview attempt status
supabase
  .channel(`attempt-${attemptId}`)
  .on('postgres_changes', {
    event: 'UPDATE',
```

```
    schema: 'public',
    table: 'interview_attempts',
    filter: `id=eq.${attemptId}`
  }, (payload) => {
    // Update UI on status change
  })
  .subscribe();
```

6. Core Flow Sequence Diagrams

6.1 Interview Creation & Question Generation Flow

```
sequenceDiagram
    participant HR as HR Recruiter
    participant CI as CreateInterview
    participant EF as Edge Function
    participant AI as Gemini AI
    participant DB as Database

    HR->>CI: Fill form: title, JD,<br/>questions config

    Note over CI: Validate (Zod):<br/>title 5-200 chars<br/>JD 50-10k<br/>chars<br/>questionCount 5-100<br/>types sum = total<br/>topics sum = 100%

    CI->>DB: INSERT interview
    DB-->>CI: interview.id

    CI->>EF: POST /functions/v1/<br/>generate-questions

    Note over EF: Build prompt:<br/>JD + config +<br/>type distribution

    EF->>AI: POST /v1/models/<br/>gemini-2.5-flash
    AI-->>EF: Structured JSON<br/>questions array

    Note over EF: Validate & parse
    EF->>DB: INSERT questions
    DB-->>EF: OK

    EF-->>CI: { questions, count }
    CI-->>HR: Show questions for review

    style HR fill:#e1bee7,stroke:#6a1b9a
    style AI fill:#fff9c4,stroke:#f9a825
    style DB fill:#c8e6c9,stroke:#2e7d32
```

6.2 Candidate Interview Taking Flow

```
sequenceDiagram
    participant C as Candidate
```

participant TI as TakeInterview
participant EF as Edge Functions
participant DB as Database
participant PR as Proctoring

C->>TI: Open share link
TI->>EF: resolve-invitation
(3 retries, 1.5s backoff)
EF->>DB: Verify token
DB-->>EF: Token valid
EF-->>TI: { invitation, interview }

C->>TI: Enter name + email
Note over TI: Validate:
name 2-100 letters
email matches invite

alt If proctoring enabled
 C->>TI: Pre-interview checks
 Note over C,TI: PreInterviewChecks:
1. Instructions
2.
Consent
3. Setup: Camera ✓, Mic ✓,
Network ✓, Lighting ✓, Person ✓
4.
Ready
end

TI->>DB: RPC: create_interview_attempt
DB-->>TI: { attempt_id, session_token }
64-char token

TI->>EF: init-proctoring
EF-->>TI: OK
TI->>PR: Start monitor

TI->>DB: RPC: get_questions_for_candidate
DB-->>TI: Questions (no correct answers!)

Note over C: ⌚ TIMER ON

Note over C: Answer questions
(MCQ, coding, scenario, descriptive)
Note over PR: Monitoring:
tab switch, copy detect,
typing patterns

C->>TI: Submit / Timer expires
Note over TI: Validate:
sessionToken ≥32ch
answers valid
timeTaken
0-86400

TI->>DB: RPC: update_attempt_with_session
DB-->>TI: OK

TI->>EF: auto-evaluate-trigger
Note over EF: evaluate-interview (async)

TI-->>C: Redirect to /interview-complete/:id

style C fill:#e1bee7,stroke:#6a1b9a
style PR fill:#ffcdd2,stroke:#c62828
style DB fill:#c8e6c9,stroke:#2e7d32

6.3 Authentication Flow (Detailed)

```
sequenceDiagram
    participant U as User
    participant AP as Auth Page
    participant GT as GoTrue
    participant EF as Edge Function
    participant DB as Database

    U->>AP: Enter email
    AP->>DB: RPC: check_user_exists
    DB-->>AP: exists? (boolean)

    alt User EXISTS (Sign In)
        AP-->>U: Show password field
        U->>AP: Enter password
        AP->>GT: POST /auth/v1/token<br/>?grant_type=password
        Note over GT: Verify bcrypt<br/>Generate JWT
        GT-->>AP: JWT + session
    else New User (Sign Up)
        AP-->>U: Show signup form
        U->>AP: Enter name + password
        AP->>GT: POST /auth/v1/signup
        Note over GT: Create auth.user<br/>Send verify email
        GT-->>AP: OK
        AP->>EF: complete-user-signup
        EF->>DB: Create profile
        EF->>DB: Assign guest role
        EF->>DB: Init onboarding
        EF->>DB: Send welcome
        DB-->>EF: OK
        EF-->>AP: Signup complete
        AP-->>U: "Check your email"
    end

    style U fill:#e1bee7,stroke:#6a1b9a
    style GT fill:#fff9c4,stroke:#f9a825
    style DB fill:#c8e6c9,stroke:#2e7d32
```

6.4 Learning Assessment Flow

```
sequenceDiagram
    participant U as User
    participant LD as LearningDash
    participant TC as TakeCertification
    participant EF as Edge Function
    participant DB as Database

    U->>LD: Select certification
    LD->>DB: can_user_retake_certification
```

```
DB-->>LD: allowed: true

LD-->>U: Navigate to /take-cert/:id

Note over U: PreInterviewChecks:<br/>Instructions → Consent → Setup
-><br/>Camera/Mic/Network/Lighting/Person

LD->>TC: Create attempt
TC->>DB: INSERT attempt
DB-->>TC: attempt_id

TC->>EF: generate-cert-questions
Note over EF: Call Gemini AI
EF-->>TC: Generated questions

TC-->>U: Show questions + timer

Note over U: Answer MCQ +<br/>coding + descriptive

U->>TC: Submit
TC->>EF: evaluate-cert
Note over EF: score ≥ passing_score<br/>→ issue certificate
EF-->>TC: Evaluation result

TC-->>U: Show result (/cert-result)

style U fill:#e1bee7,stroke:#6a1b9a
style EF fill:#fff9c4,stroke:#f9a825
style DB fill:#c8e6c9,stroke:#2e7d32
```

7. State Management Detail

7.1 AuthContext Implementation

```
// src/contexts/AuthContext.tsx

interface AuthState {
  user: User | null;
  session: Session | null;
  loading: boolean;
  roles: AppRole[];
}

interface AuthContextValue extends AuthState {
  signIn: (email: string, password: string) => Promise<void>;
  signUp: (email: string, password: string, fullName: string) => Promise<void>;
  signOut: () => Promise<void>;
  hasRole: (role: AppRole) => boolean;
  hasAnyRole: (roles: AppRole[]) => boolean;
}
```

```
// State machine:
//
// INITIAL → loading=true, user=null
//   ↓ onAuthStateChange fires
// SIGNED_IN → loading=false, user=User, session=Session
//   ↓ ensureUserSetup()
//     ↓ fetchUserRoles() (setTimeout 0 to avoid deadlock)
// READY → roles populated, redirects active
//
// SIGNED_OUT → loading=false, user=null, roles=[]

// God mode: hasAnyRole() returns true if user is platform_admin
// regardless of the requested roles
```

7.2 OrganizationContext Implementation

```
// src/contexts/OrganizationContext.tsx

interface OrgContextValue {
  selectedOrgId: string | null;
  selectedOrg: Organization | null;
  availableOrgs: Organization[];
  isImpersonating: boolean; // platform_admin viewing another org
  selectOrganization: (id: string) => void;
  clearImpersonation: () => void;
}

// Behavior:
// - platform_admin: Can select any org (OrganizationSelector component)
// - partner_admin: Locked to their org (auto-selected from org_members)
// - other roles: Linked to org via organization_members
```

7.3 React Query Usage Patterns

```
// Data fetching pattern (used across all pages)

// List query with filters
const { data: interviews, isLoading } = useQuery({
  queryKey: ['interviews', orgId, status, page],
  queryFn: async () => {
    const { data, error } = await supabase
      .from('interviews')
      .select('*')
      .eq('organization_id', orgId)
      .eq('status', status)
      .order('created_at', { ascending: false })
      .range(page * 10, (page + 1) * 10 - 1);
    if (error) throw error;
  }
});
```

```

        return data;
    },
    enabled: !!orgId, // Only fetch when org is selected
  });

// Mutation with optimistic update
const updateMutation = useMutation({
  mutationFn: async (data: Partial<Interview>) => {
    const { error } = await supabase
      .from('interviews')
      .update(data)
      .eq('id', interviewId);
    if (error) throw error;
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['interviews'] });
    toast.success('Interview updated');
  },
  onError: (error) => {
    toast.error(getUserFriendlyErrorMessage(error));
  },
});

// Edge function invocation
const generateMutation = useMutation({
  mutationFn: async () => {
    return invokeFunction('generate-questions', {
      interviewId,
      config: formData,
    });
  },
});

```

7.4 Custom Hooks Design

```

// src/hooks/useUserRoles.ts
interface UseUserRolesReturn {
  roles: AppRole[];
  loading: boolean;
  isPlatformAdmin: boolean;
  isPartnerAdmin: boolean;
  isHRRecruiter: boolean;
  isTechSpoc: boolean;
  isBillingContact: boolean;
  isGuest: boolean;
  hasRole: (role: AppRole) => boolean;
  hasAnyRole: (roles: AppRole[]) => boolean;
  refetchRoles: () => Promise<void>;
}

// src/hooks/usePermissions.ts

```

```

interface UsePermissionsReturn {
  hasPermission: (permission: Permission) => boolean;
  hasAnyPermission: (permissions: Permission[]) => boolean;
  hasAllPermissions: (permissions: Permission[]) => boolean;
  userPermissions: Permission[];
}
// Maps roles → permissions via ROLE_PERMISSIONS constant in permissions.ts

// src/hooks/useProctoring.ts
interface UseProctoringReturn {
  isActive: boolean;
  violations: Violation[];
  tabSwitchCount: number;
  lookAwayCount: number;
  integrityScore: number;
  startMonitoring: () => void;
  stopMonitoring: () => void;
  logViolation: (type: ViolationType, data?: object) => void;
  getReport: () => ProctoringReport;
}

```

8. Component Design

8.1 Interview Creation Form Architecture

```

flowchart TD
    CI["CreateInterview (Page)"]

    CI --> BI["InterviewBasicInfo"]
    CI --> QT["QuestionTypeInputs"]
    CI --> CD["CategoryDifficultyInputs"]
    CI --> TD2["TopicDistribution"]
    CI --> RQ["RequiredQuestionRules"]
    CI --> PT["ProctoringToggle"]
    CI --> AB["ActionButtons"]

    BI --> BI1["Input: title<br/>(5-200 chars)"]
    BI --> BI2["Textarea: jobDescription<br/>(50-10,000 chars)"]
    BI --> BI3["Input: questionCount<br/>(5-100)"]
    BI --> BI4["Input: timeLimit<br/>(0-180 min)"]

    QT --> QT1["Input: mcqCount"]
    QT --> QT2["Input: scenarioCount"]
    QT --> QT3["Input: codingCount"]
    QT --> QT4["Input: descriptiveCount"]
    QT --> QTV["Validation: sum must<br/>equal questionCount"]

    CD --> CDS["Per-type difficulty sliders"]
    CDS --> CDE["Easy %"]
    CDS --> CDM["Medium %"]

```

```

CDS --> CDH["Hard %"]
CD --> CDV["Validation: each must<br/>sum to 100%"]

TD2 --> TD2A["Extracted skills<br/>(from JD via AI)"]
TD2 --> TD2B["Percentage per topic slider"]
TD2 --> TD2V["Validation: must sum to 100%"]

RQ --> RQR["Rule builder: at least N<br/>{difficulty} {type} on {topic}"]

PT --> PTS["Switch: proctoring_enabled"]

AB --> AB1["Save Draft"]
AB --> AB2["Generate Questions<br/>(disabled when form invalid)"]
AB --> AB3["Cancel"]

style CI fill:#bbdefb,stroke:#1565c0
style QTV fill:#fff9c4,stroke:#f9a825
style CDV fill:#fff9c4,stroke:#f9a825
style TD2V fill:#fff9c4,stroke:#f9a825
style AB2 fill:#c8e6c9,stroke:#2e7d32

```

8.2 Proctoring Component Architecture

flowchart TD

```
PIC["PreInterviewChecks"]
```

```

PIC --> S1["Step 1: InstructionsStep"]
S1 --> S1A["Rules list, continue button"]

```

```

PIC --> S2["Step 2: ConsentStep"]
S2 --> S2A["Checkbox: Allow proctoring"]
S2 --> S2B["Checkbox: Allow recording"]
S2 --> S2C["Checkbox: Allow data usage"]
S2 --> S2D["Checkbox: Allow AI analysis"]
S2 --> S2V["All required to proceed"]

```

```

PIC --> S3["Step 3: SetupStep"]
S3 --> NET["NetworkCheck<br/>5 x 1MB samples → median bandwidth<br/>Pass: ≥1 Mbps, Warn: <5 Mbps"]

```

```

S3 --> CAM["CameraCheck<br/>getUserMedia(640x480) → validate stream"]
S3 --> MIC["MicrophoneCheck<br/>Audio track presence check"]
S3 --> LIT["LightingCheck<br/>checkCameraQuality() + brightness analysis"]
S3 --> PER["PersonCheck<br/>detectFaces() → exactly 1 person required"]

```

```

PIC --> S4["Step 4: ReadyStep"]
S4 --> S4A["Confirmation to start"]

```

```
PM["ProctoringMonitor<br/>(runs during assessment)"]
```

```

PM --> PM1["useCandidateBroadcast → live stream"]
PM --> PM2["FaceDetection → personCount, confidence"]

```

```

PM --> PM3["GazeTracking → eye gaze coordinates"]
PM --> PM4["VoiceAnalysis → voice count, anomalies"]

PM --> PM5["TabSwitchDetector → visibilitychange API"]
PM --> PM6["CopyPasteDetector → clipboard event intercept"]
PM --> PM7["PrintScreenBlocker → keyboard event intercept"]
PM --> PM8["MultiMonitorDetector → screen enumeration"]
PM --> PM9["VMDetector → browser fingerprint"]
PM --> PM10["TypingPatternAnalyzer → keystroke dynamics"]

PM --> PM11["ScreenShareBlockingOverlay"]
PM --> PM12["StreamHealthGuard → auto-recovery"]
PM --> PM13["NetworkStatusBanner → connection quality"]
PM --> PM14["ChunkUploader → periodic video uploads"]

style PIC fill:#bbdefb,stroke:#1565c0
style PM fill:#ffcdd2,stroke:#c62828
style S2V fill:#fff9c4,stroke:#f9a825
style S3 fill:#e8f5e9,stroke:#2e7d32

```

8.3 UI Component Library (shadcn/ui)

flowchart TD

```

ROOT["src/components/ui/"]

ROOT --> LC["Layout & Container"]
ROOT --> FC["Form Controls"]
ROOT --> DI["Display"]
ROOT --> NAV["Navigation"]
ROOT --> FB["Feedback"]

LC --> LC1["card.tsx<br/>(Card, CardHeader, CardContent, CardFooter)"]
LC --> LC2["dialog.tsx<br/>(Dialog, DialogContent, DialogHeader, DialogFooter)"]
LC --> LC3["sheet.tsx<br/>(Sheet – slide-over panel)"]
LC --> LC4["tabs.tsx<br/>(Tabs, TabsList, TabsTrigger, TabsContent)"]
LC --> LC5["accordion.tsx<br/>(Accordion, AccordionItem, AccordionContent)"]
LC --> LC6["separator.tsx<br/>(Visual separator)"]

FC --> FC1["button.tsx<br/>(default|destructive|outline|secondary|ghost|link)"]
FC --> FC2["input.tsx<br/>(text, email, password, number)"]
FC --> FC3["textarea.tsx<br/>(multi-line text)"]
FC --> FC4["select.tsx<br/>(single-select dropdown)"]
FC --> FC5["checkbox.tsx / switch.tsx / slider.tsx"]
FC --> FC6["calendar.tsx / form.tsx / command.tsx"]

DI --> DI1["table.tsx<br/>(Table, TableHeader, TableBody, TableRow, TableCell)"]
DI --> DI2["badge.tsx<br/>(status/tag badges)"]
DI --> DI3["avatar.tsx / alert.tsx"]
DI --> DI4["progress.tsx / skeleton.tsx / tooltip.tsx"]

NAV --> NAV1["dropdown-menu.tsx<br/>(context menus)"]

```

```
NAV --> NAV2["popover.tsx<br/>(positioned popover)"]
NAV --> NAV3["sidebar.tsx<br/>(collapsible sidebar)"]
NAV --> NAV4["breadcrumb.tsx<br/>(nav breadcrumbs)"]
```

```
FB --> FB1["toast.tsx<br/>(Sonner-based toasts)"]
FB --> FB2["sonner.tsx<br/>(toast provider)"]
```

```
style ROOT fill:#bbdefb,stroke:#1565c0
style LC fill:#e8f5e9,stroke:#2e7d32
style FC fill:#fff3e0,stroke:#e65100
style DI fill:#f3e5f5,stroke:#6a1b9a
style NAV fill:#e0f7fa,stroke:#00695c
style FB fill:#fce4ec,stroke:#c62828
```

9. Security Implementation Detail

9.1 RLS Policy Patterns

```
-- Pattern 1: User owns the resource
CREATE POLICY "users_own_data" ON profiles
  FOR ALL USING (id = auth.uid());

-- Pattern 2: Organization-scoped access
CREATE POLICY "org_member_access" ON interviews
  FOR SELECT USING (
    organization_id IN (
      SELECT organization_id FROM organization_members
      WHERE user_id = auth.uid() AND is_active = true
    )
    OR has_role(auth.uid(), 'platform_admin')
  );

-- Pattern 3: Role-restricted write
CREATE POLICY "hr_can_create" ON interviews
  FOR INSERT WITH CHECK (
    has_any_role(auth.uid(), ARRAY['platform_admin', 'partner_admin',
    'hr_recruiter', 'tech_spoc'])
  );

-- Pattern 4: Candidate-only access (no auth required)
CREATE POLICY "candidate_read_attempt" ON interview_attempts
  FOR SELECT USING (
    session_token IS NOT NULL
    -- Validated in SECURITY DEFINER function
  );

-- Pattern 5: Immutable fields
CREATE POLICY "attempt_update_restrict" ON interview_attempts
  FOR UPDATE WITH CHECK (
    candidate_email = (SELECT candidate_email FROM interview_attempts WHERE id = id)
```

```

    AND candidate_name = (SELECT candidate_name FROM interview_attempts WHERE id =
id)
);

```

9.2 Error Sanitization Pipeline

```

flowchart TD
    PGE["PostgreSQL Error"] --> EH

    subgraph EH["error-handler.ts"]
        direction TB
        CODE["PostgreSQL Code Mapping"]
        CODE --> C1["23505 → This record already exists"]
        CODE --> C2["42501 → Permission denied"]
        CODE --> C3["PGRST116 → Record not found"]
        CODE --> C4["PGRST301 → No rows returned"]
        CODE --> C5["57P01 → Service temporarily down"]
        CODE --> C6["42P01 → Configuration error"]

        CTX["Context-specific handlers"]
        CTX --> H1["getAuthErrorMessage(error)"]
        CTX --> H2["getInterviewErrorMessage(error)"]
        CTX --> H3["getUserFriendlyErrorMessage(error)"]

        SEC["NEVER expose: table names,<br/>column names, SQL queries,<br/>stack
traces, internal IPs"]
    end

    EH --> TOAST["User-Friendly Toast Message"]

    style PGE fill:#ffcd2,stroke:#c62828
    style EH fill:#fff3e0,stroke:#e65100
    style TOAST fill:#c8e6c9,stroke:#2e7d32
    style SEC fill:#ffcd2,stroke:#c62828

```

9.3 Token Flow

```

flowchart TD
    subgraph JWT["Session JWT"]
        direction TB
        J1["Created: GoTrue on sign-in"]
        J2["Storage: Browser (managed by Supabase client)"]
        J3["Lifetime: Configurable (default 1 hour)"]
        J4["Refresh: Auto-refresh with 60s buffer"]
        J5["Claims: { sub: user_id, role: authenticated,<br/>email, iat, exp }"]
        J6["Used by: All PostgREST queries, edge function calls"]
        J7["RLS reads: auth.uid<#40;&#41; extracts sub from JWT"]
    end
    end

```

```

subgraph IST["Interview Session Token"]
    direction TB
    I1["Created: create_interview_attempt<#40;#41; PG function"]
    I2["Value: 64-char base64 (gen_random_bytes(48))"]
    I3["Storage: Browser sessionStorage (not localStorage)"]
    I4["Lifetime: Duration of interview attempt"]
    I5["Used by: update_attempt_with_session<#40;#41; to validate"]
    I6["Purpose: Prevents unauthorized answer submission"]
    I7["Cannot: Be guessed or enumerated"]
end

subgraph API["API Keys (AI Providers)"]
    direction TB
    A1["Storage: ai_provider_credentials table"]
    A2["Encryption: AES via encrypt_api_key<#40;#41; function"]
    A3["Decryption: Only in SECURITY DEFINER functions"]
    A4["Access: Edge functions only (service role)"]
    A5["Never: Sent to client or exposed in API responses"]
end

style JWT fill:#bbdefb,stroke:#1565c0
style IST fill:#fff9c4,stroke:#f9a825
style API fill:#ffcdd2,stroke:#c62828

```

10. Error Handling Architecture

10.1 Error Boundary Hierarchy

```

flowchart TD
    APP["App"]
    ROOT["ErrorBoundary<br/>section=root<br/>fallback=RootErrorPage"]
    APP --> ROOT

    ROOT --> AL["AdminLayout"]
    AL --> AEB["ErrorBoundary<br/>section=admin<br/>fallback=AdminErrorPage"]
    AEB --> AR["Route component"]

    ROOT --> PL["PartnerLayout"]
    PL --> PEB["ErrorBoundary<br/>section=partner<br/>fallback=PartnerErrorPage"]
    PEB --> PR["Route component"]

    ROOT --> RR["Recruiting routes"]
    RR --> REB["ErrorBoundary<br/>section=recruiting<br/>fallback=RecruitingErrorPage"]
    REB --> RRC["Route component"]

    ROOT --> ASR["Assessment routes"]
    ASR --> ASEB["ErrorBoundary<br/>section=assessment<br/>fallback=AssessmentErrorPage"]
    ASEB --> ASC["Route component"]

```

NOTE["Each ErrorBoundary:
1. Catches React render errors
2. Reports to Sentry with section tag
3. Shows fallback UI with retry option
4. Logs to console in development"]

```
style APP fill:#bbdefb,stroke:#1565c0
style ROOT fill:#ffcdd2,stroke:#c62828
style AEB fill:#fff3e0,stroke:#e65100
style PEB fill:#fff3e0,stroke:#e65100
style REB fill:#fff3e0,stroke:#e65100
style ASEB fill:#fff3e0,stroke:#e65100
style NOTE fill:#f5f5f5,stroke:#9e9e9e
```

10.2 Retry Strategy

```
// src/lib/retryUtils.ts

interface RetryConfig {
  maxAttempts: number; // Default: 3
  initialDelay: number; // Default: 1000ms
  maxDelay: number; // Default: 5000ms
  backoffMultiplier: number; // Default: 1.5
}

function isRetryable(error: Error): boolean {
  // Retryable:
  // - Network errors (TypeError: Failed to fetch)
  // - Timeout errors
  // - HTTP 500, 502, 503, 504
  // - HTTP 429 (rate limited)

  // NOT retryable:
  // - HTTP 400, 401, 403, 404, 409
  // - Validation errors
  // - Auth errors
}

// Retry flow:
// Attempt 1 → fail → wait 1.0s
// Attempt 2 → fail → wait 1.5s
// Attempt 3 → fail → throw error
```

10.3 Logging Layers

```
flowchart TD
    subgraph ARCH["LOGGING ARCHITECTURE"]
        direction TB

        subgraph L1["Layer 1: logger.ts (General purpose)"]
```

```

    L1A["logger.debug / info / warn / error"]
    L1B["Output: Console (dev), Sentry (prod)"]
end

subgraph L2["Layer 2: systemLogger.ts (System events)"]
    L2A["systemLogger.logEvent(category, action, data)"]
    L2B["Categories: auth, navigation, api, error"]
    L2C["Output: console + optional persistence"]
end

subgraph L3["Layer 3: operationLogger.ts (Business operations)"]
    L3A["const op = startOperation(submit_interview)"]
    L3B["op.complete({ attemptId }) or op.fail(error)"]
    L3C["Tracks: duration, success/failure, metadata"]
end

subgraph L4["Layer 4: audit-logger.ts (Admin audit trail)"]
    L4A["auditLog(delete_user, { userId, performedBy })"]
    L4B["Persisted to: audit_logs table"]
    L4C["Retention: 90 days"]
end

end

style ARCH fill:#f5f5f5,stroke:#616161
style L1 fill:#bbdefb,stroke:#1565c0
style L2 fill:#c8e6c9,stroke:#2e7d32
style L3 fill:#fff9c4,stroke:#f9a825
style L4 fill:#ffcdd2,stroke:#c62828

```

11. Proctoring Engine Detail

11.1 Violation Detection Pipeline

```

flowchart TD
    subgraph PIPELINE["PROCTORING DETECTION PIPELINE"]
        direction TB

        subgraph RT["REAL-TIME (During Interview)"]
            direction TB
            EL["Event Listeners"]
            EL --> EL1["document.visibilitychange → tab_switch"]
            EL --> EL2["document.copy/paste → copy_attempt"]
            EL --> EL3["keyboard.PrintScreen → print_screen"]
            EL --> EL4["keystroke timing → suspicious_typing"]

            AC["API Checks"]
            AC --> AC1["screen.availWidth > window.innerWidth<br/>→ multiple_monitors"]
            AC --> AC2["navigator.userAgent patterns<br/>→ virtual_machine"]
            AC --> AC3["MediaStream.onended<br/>→ screen_share_stopped"]
        end
    end

```

```

end

subgraph DEF["DEFERRED (Post-Interview Analysis)"]
    direction TB
    MP["MediaPipe Tasks Vision"]
    MP --> FD["Face Detection:<br/>personCount > 1 → multiple_persons (HIGH)"]
    FD --> C["confidence < threshold → look_away (LOW)"]
    MP --> GT["Gaze Tracking:<br/>off-screen > 3s → look_away (LOW)"]
    GT --> EP["erratic pattern → eye_movement (LOW)"]

    VA["Voice Analysis"]
    VA --> VA1["Multiple voice profiles → multiple_voices (MED)"]
    VA --> VA2["Silence patterns → silence_anomaly (LOW)"]
    VA --> VA3["TTS/recording detected → audio_playback (MED)"]

    AD["Advanced Detection"]
    AD --> AD1["Face mismatch → person_swap (HIGH)"]
    AD --> AD2["Liveness check fail → liveness_fail (HIGH)"]
    AD --> AD3["Object detection → phone_detected (HIGH)"]
    AD --> AD4["Object detection → prohibited_object (HIGH)"]
end

subgraph SCORE["Severity Scoring"]
    SC1["HIGH violations: -10 points each"]
    SC2["MEDIUM violations: -5 points each"]
    SC3["LOW violations: -2 points each"]
    SC4["Integrity Score = 100 - sum(penalties); Minimum: 0"]
end

end

style PIPELINE fill:#f5f5f5,stroke:#616161
style RT fill:#bbdefb,stroke:#1565c0
style DEF fill:#fff3e0,stroke:#e65100
style SCORE fill:#ffcdd2,stroke:#c62828

```

11.2 Recording Architecture

```

flowchart TD
    subgraph RECORD["RECORDING & UPLOAD PIPELINE"]
        direction TB
        CS["Camera Stream"] --> MR["MediaRecorder (WebM)"]
        MR -->|"Chunk every 30s"| CU["ChunkUploader"]
        CU --> GU["get-chunk-upload-url<br/>(signed URL)"]
        GU --> US["Upload to Storage<br/>(background, retryable)"]

        MR -->|"On Interview Complete"| MERGE["merge-proctoring-chunks"]
        MERGE --> REPAIR["repair-webm-metadata"]
        REPAIR --> CONFIRM["confirm-proctoring-upload"]
        CONFIRM --> FINAL["proctoring_sessions.video_url = final URL<br/>upload_status = complete"]
    end

```

```

    SS["Screen Share Stream<br/>(same pipeline)"] --> MR
end

subgraph GUARD["Upload Guard"]
    direction TB
    UL["uploadLock.ts:<br/>• Prevents duplicate uploads<br/>• Tracks pending
chunks<br/>• Retries failed chunks (3 attempts)"]
    UP["uploadProgressStore.ts:<br/>• Progress percentage per chunk<br/>• Total
progress calculation<br/>• UI progress bar binding"]
    BG["BackgroundUploadHandler.tsx:<br/>• Resumes interrupted uploads<br/>•
Runs as app-level component<br/>• Shows UploadProgressOverlay"]
end

style RECORD fill:#bbdefb,stroke:#1565c0
style GUARD fill:#fff3e0,stroke:#e65100
style FINAL fill:#c8e6c9,stroke:#2e7d32

```

12. File Upload Architecture

12.1 Upload Flow

```

sequenceDiagram
    participant C0 as Component
    participant CU as ChunkUploader
    participant EF as Edge Function
    participant ST as Storage

    C0->>CU: Start recording

    loop Every 30 seconds
        C0->>CU: dataavailable event
        CU->>EF: get-chunk-upload-url
        EF-->>CU: { signedUrl, path }
        CU->>ST: PUT signedUrl (blob)
        ST-->>CU: 200 OK
        CU-->>C0: progress update
    end

    Note over C0: On completion
    CU->>EF: merge-proctoring-chunks
    EF->>ST: Merge to single file
    ST-->>EF: OK
    EF-->>CU: { videoUrl }

    style C0 fill:#bbdefb,stroke:#1565c0
    style ST fill:#c8e6c9,stroke:#2e7d32

```

13. Configuration Management

13.1 Environment Variables

```
flowchart TD
    subgraph FE["Frontend (Vite - VITE_ prefix)"]
        F1["VITE_SUPABASE_URL = http://localhost:8000"]
        F2["VITE_SUPABASE_PUBLISHABLE_KEY = anon_key"]
        F3["VITE_SENTRY_DSN = sentry_dsn"]
        F4["VITE_APP_ENV = development | production"]
    end

    subgraph BE["Backend (Docker environment)"]
        B1["POSTGRES_PASSWORD = db_password"]
        B2["JWT_SECRET = jwt_signing_secret"]
        B3["ANON_KEY / SERVICE_ROLE_KEY"]
        B4["GOTRUE_SMTP_HOST / PORT / USER / PASS"]
        B5["GOTRUE_EXTERNAL_EMAIL_ENABLED = true"]
        B6["GOOGLE_AI_API_KEY = gemini_key"]
        B7["STRIPE_SECRET_KEY = stripe_key"]
    end

    style FE fill:#bbdefb,stroke:#1565c0
    style BE fill:#fff3e0,stroke:#e65100
```

13.2 Platform Configuration System

```
flowchart TD
    subgraph HIER["CONFIGURATION HIERARCHY"]
        direction TB
        P1["1. Environment Variables<br/>(highest priority)"]
        P1 --> P1A["Build-time: VITE_* variables"]
        P1 --> P1B["Runtime: Docker env vars"]

        P2["2. Database Configuration"]
        P2 --> P2A["platform_configurations<br/>(key-value pairs)"]
        P2 --> P2B["system_config<br/>(system-level settings)"]
        P2 --> P2C["ai_feature_configurations<br/>(AI feature flags)"]

        P3["3. Organization Settings"]
        P3 --> P3A["organizations.settings<br/>(JSONB per-org config)"]

        P4["4. Code Defaults"]
        P4 --> P4A["src/lib/configuration.ts<br/>(default values)"]
        P4 --> P4B["src/lib/configurationPresets.ts<br/>(presets)"]

        P1 ~~~ P2
        P2 ~~~ P3
        P3 ~~~ P4
    end

    style P1 fill:#ffcdd2,stroke:#c62828
```

```
style P2 fill:#fff3e0,stroke:#e65100
style P3 fill:#fff9c4,stroke:#f9a825
style P4 fill:#c8e6c9,stroke:#2e7d32
```

14. Testing Architecture

14.1 Test Stack

```
flowchart TD
    subgraph ARCH["TESTING ARCHITECTURE"]
        direction TB

        subgraph E2E["E2E Tests (Playwright)"]
            direction TB
            EC["Config: playwright.config.ts<br/>Browser: Chromium only<br/>Workers: 1 (sequential)<br/>Timeout: 60s per test<br/>Base URL: http://localhost:5174"]
            EF["Test Files:<br/>e2e/11-rbac through 18-profile<br/>e2e/flow-01 through flow-18"]
            ES["Total: 630 tests, all passing<br/>Runtime: ~1 hour (sequential)"]
            EH["Helpers:<br/>helpers.ts, auth-utils.ts, globalSetup.ts"]
        end

        subgraph UNIT["Unit Tests (Vitest)"]
            UC["Config: vitest.config.ts<br/>Coverage: lcov reporting<br/>Scope: Utility functions, validators, helpers"]
        end

        subgraph SEED["Test Data Seeding"]
            direction TB
            SU["6 Test Users:<br/>e2e-admin (platform_admin)<br/>e2e-partner (partner_admin)<br/>e2e-hr (hr_recruiter)<br/>e2e-tech (tech_spoc)<br/>e2e-billing (billing_contact)<br/>e2e-guest (guest)"]
            SD["Seeded via globalSetup:<br/>Organization, Interviews,<br/>Subscription plans,<br/>Notifications, Training data"]
        end

        style ARCH fill:#f5f5f5,stroke:#616161
        style E2E fill:#bbdefb,stroke:#1565c0
        style UNIT fill:#c8e6c9,stroke:#2e7d32
        style SEED fill:#fff9c4,stroke:#f9a825
    end
```

14.2 Test Categories

```
flowchart TD
    subgraph MATRIX["TEST COVERAGE MATRIX"]
        direction TB
        R1["RBAC/Access Control – 11-rbac – ~50 tests – Security"]
        R2["Platform Admin Deep – 12-admin – ~40 tests – Feature"]
    end
```

```
R3["Partner Admin Deep - 13-partner - ~30 tests - Feature"]
R4["HR Recruiter Deep - 14-hr - ~35 tests - Feature"]
R5["Tech SPOC Deep - 15-tech - ~25 tests - Feature"]
R6["Billing Contact Deep - 16-billing - ~25 tests - Feature"]
R7["Guest User Deep - 17-guest - ~30 tests - Feature"]
R8["Profile/Notifications - 18-profile - ~30 tests - Feature"]
R9["Flow Tests 01-12 - flow-01-12 - 98 tests - E2E Flow"]
R10["Profile CRUD - flow-13 - 9 tests - CRUD"]
R11["Interview CRUD - flow-14 - 13 tests - CRUD"]
R12["User Management CRUD - flow-15 - 11 tests - CRUD"]
R13["Form Validation - flow-16 - 12 tests - Validation"]
R14["Data Persistence - flow-17 - 21 tests - API"]
R15["Candidate Journey - flow-18 - 23 tests - E2E Flow"]
TOTAL["TOTAL: 26 files - 630 tests"]
end
```

```
style MATRIX fill:#f5f5f5,stroke:#616161
style R1 fill:#ffcdd2,stroke:#c62828
style R9 fill:#bbdefb,stroke:#1565c0
style R15 fill:#bbdefb,stroke:#1565c0
style TOTAL fill:#c8e6c9,stroke:#2e7d32
```